СХЕМОТЕХНИКА.
ЛАБОРАТОРНАЯ РАБОТА 4.
ВВЕДЕНИЕ В ПРОЕКТИРОВАНИЕ ЦИФРОВЫХ СХЕМ НА VERILOG С ИСПОЛЬЗОВАНИЕМ ТАБЛИЦЫ ИСТИННОСТИ

1. Цели лабораторной работы

Из предыдущей лабораторной работе вы узнали, что Verilog - это быстрый и простой способ создания цифровых схем в Altera Quartus II. Рассмотрим другой способ реализации проекта: использование таблицы истинности.

В этой лабораторной работе вы научитесь реализовывать таблицу истинности, используя Verilog.

- В первой части лабораторной работы вы изучите реализацию таблицы истинности на языке Verilog.
- Во второй части вы будете использовать интегрированный подход: схематический ввод, Verilog и таблицы истинности, чтобы выполнить смешанное проектирование.

2. Предварительная подготовка

Создайте на устройстве, где храните свои проекты, новую папку **…\Lab04**, где будете сохранять файлы по этой лабораторной работе.

3. Реализация таблицы истинности с использованием Verilog (Lab4_1)

Один из способов определить таблицу истинности в Verilog - использовать оператор **case**. Существуют и другие способы определения таблицы истинности, но в этой лабораторной работе мы будем использовать оператор **case**. Рассмотрим пример ввода следующей таблицы истинности в Verilog:

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

```
module truth_table_verilog_example (F, A, B, C);
	input A, B, C;
	output F;
	reg F; // procedural statements can only change the values of
		// variables declared as memory elements such as a reg (register)

	always @(A or B or C)			// invoke procedural code when these change
	begin					// begin the procedural statements
		case ({A,B,C})			// variables that affect the procedure

						// the 3'b simply means that we are
			3'b000: F= 'b0;		// considering a three-bit binary value
			3'b001: F= 'b0;
			3'b010: F= 'b1;
			3'b011: F= 'b0;
			3'b100: F= 'b1;
			3'b101: F= 'b0;
			3'b110: F= 'b1;
			3'b111: F= 'b0;
		endcase
	end	//end the begin statement
endmodule
```

Теперь создайте новый проект и напишите код для задачи фермера (включая фермера, смотри

Lab03_3), используя метод таблиц истинности в Verilog. Постановку задачи фермера смотрите в лабораторной работе 3.
Затем выполните симуляцию проекта. Сформируйте входные сигналы следующим образом:

- Сигнал **F**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 400 ns. Данная последовательность должна периодически повторяться с периодом 800 ns до конца времени моделирования.
- Сигнал **C**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 200 ns. Данная последовательность должна периодически повторяться с периодом 400 ns до конца времени моделирования.
- Сигнал **G**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 100 ns. Данная последовательность должна периодически повторяться с периодом 200 ns до конца времени моделирования.
- Сигнал **W**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 50 ns. Данная последовательность должна периодически повторяться с периодом 100 ns до конца времени моделирования.

Прежде чем двигаться дальше вам необходимо добиться, чтобы ваш код работал на ModelSim. Убедитесь в правильности формы выходного сигнала.

4. Смешанное проектирование
Теперь, когда мы знаем, как проектировать с использованием таблиц истинности, мы попробуем интегрировать все изученные до сих пор методы проектирования (схемное проектирование, с помощью Verilog и таблицы истинности) в одну единственную задачу. Проблема описана ниже.

Описание:
В доме есть кондиционер (air-conditioner - A/C), который может работать в двух режимах: в нормальном (штатном) или энергосберегающем. На рис. 1 показана общая схема системы кондиционирования. Контроллеры режимов (блоки «Нормальный режим» и «Режим энергосбережения») подключены к различным датчикам. С помощью датчиков контроллеры определяют температуру, влажность и наличие кого-либо в доме. Таким образом, входной набор: $inputs = \{Temperature, Humidity, People\}$, или просто **T,H** и **P** для краткости. Каждый из этих датчиков имеет установленное значение, и когда показания датчика падают ниже этого установленного значения, он выводит логический 0. Если показания датчика остаются выше значения, он выводит логическую 1. Например, если температура выше 25^0 C, на выходе логическая 1; в противном случае это логический 0.
Каждый контроллер получает значения датчика и выдает свой выходной сигнал, который подключается к мультиплексору 2-к-1. Мультиплексор может выбирать между двумя различными режимами. Его выход управляет кондиционером. Более подробная информация о контроллерах приведена в тексте под диаграммой. Блок-схема системы кондиционирования с датчиками и контроллерами выглядит следующим образом:

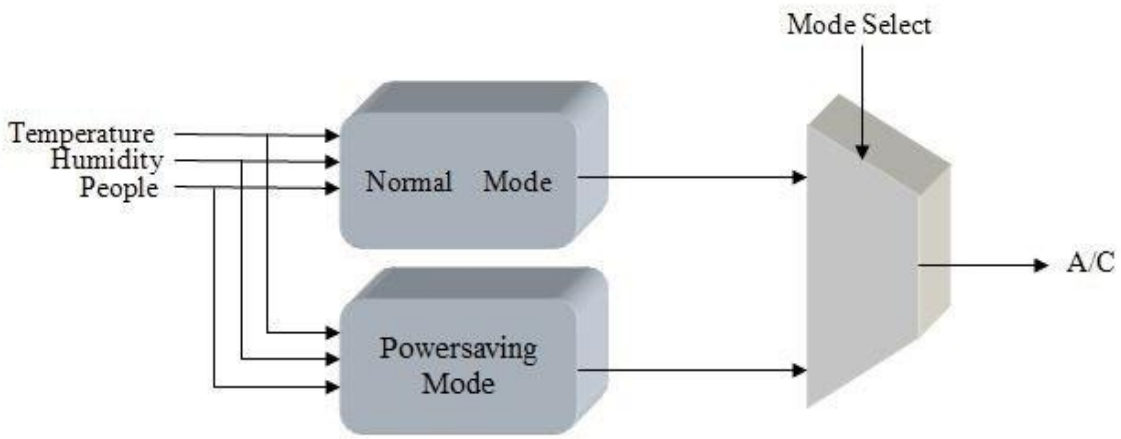


Рис 1: Общая схема системы кондиционирования

Штатный режим:

$inputs = \{Temperature, Humidity, People\}, output = \{E\}$

$E = P \cdot (T + H) + \bar{P} \cdot (T \cdot H)$ означает, что кондиционер должен быть ВКЛЮЧЕН (другими словами, имеет значение логической 1), когда кто-то присутствует (P) и либо высокая температура (T), либо высокая влажность (H), или когда никого нет в доме ($\bar{P}$), и температура и влажность высокие. Вы должны ввести этот блок в коде Verilog.

Режим энергосбережения:

$inputs = \{Temperature, Humidity, People\}, \quad output = \{F\}$

Вы должны ввести этот блок в коде Verilog. Логичное выражение: $F = P \cdot T \cdot H$ означает, что кондиционер должен быть включен, когда кто-то находится в доме, а температура и влажность высоки.

Мультиплексор:

$inputs = \{M, E, F\}, outputs = \{AC\}$

Вы должны ввести этот блок в Verilog. Когда M = 0, выбирается **штатный режим (E)**; когда M = 1, выбирается **режим энергосбережения (F)**.

- Запустите новый проект и назовите его **lab4_2**. Убедитесь, что вы запускаете новый проект в новой папке (с тем же именем). Откройте Block Diagram/Schematic file (новый файл блок-схемы) и сохраните его. Это файл графического редактора, в котором будут реализованы все компоненты системы кондиционирования.
- Затем создайте блоки, которые будут представлять компоненты кондиционера. Щелкните на **block tool** на верхней панели инструментов (см. рис. 2). Щелкните левой кнопкой мыши в любом месте пустого файла блок-диаграммы и с помощью мыши растяните и зафиксируйте блок. Повторите этот процесс еще 2 раза. Три блока будут представлять компоненты вашей системы кондиционирования. Щелкните правой кнопкой мыши на 1-й

блок и выберите «Property». Переименуйте блок в normal, затем нажмите на вкладку I/O и введите P, T, H в качестве его входов и E в качестве его выхода. Повторите те же шаги для других блоков (конечно, используя соответствующие имена входов/выходов). Убедитесь, что соглашение об именах такое же, как указано в постановке задачи выше.

- Введите 4 входных контакта (P, H, T, M) и один выходной контакт (AC) в файл проекта.
- Щелкните на **Orthogonal Conduit Tool** на верхней панели инструментов и соедините входные, выходные контакты и блоки с помощью **Conduit**. Это автоматически сопоставит выход одного блока с входом другого блока, при условии, что их имена будут совпадать. Обратитесь к рисунку ниже, чтобы убедиться, что вы сделали все правильно.

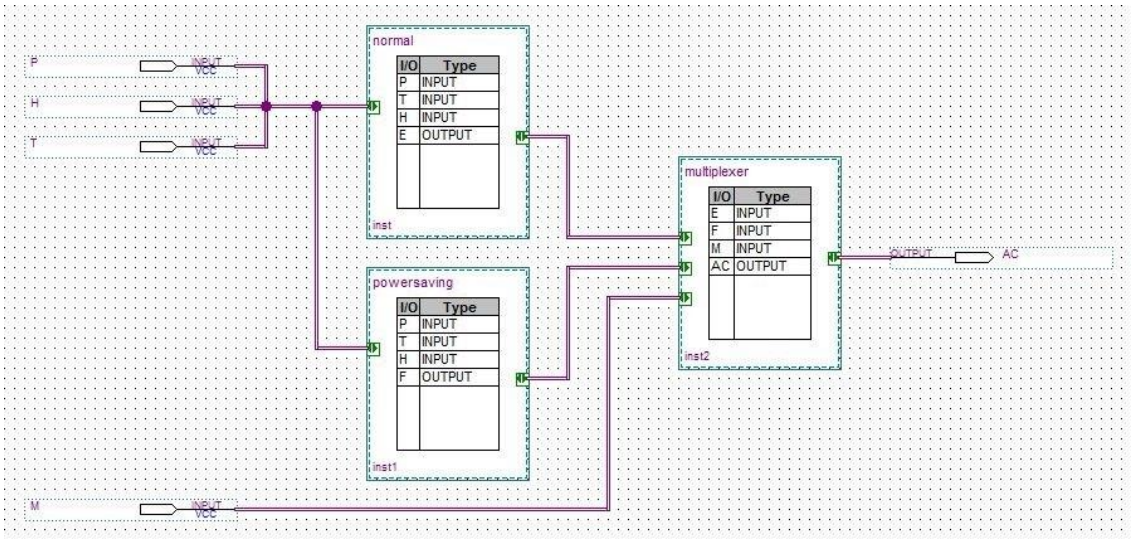


Рис. 2. Блок-схема проекта *Lab4_2*.

- Теперь, когда проект готов, напишите код для всех компонентов. Щелкните правой кнопкой мыши на блок *normal* и кликните на **Create Design File from selected Block**. Выберите Verilog и нажмите OK. Откроется файл Verilog. Напишите код для этого блока и поместите его перед оператором `endmodule`. Имейте в виду, что входы и выходы уже объявлены. Повторите этот шаг для остальных блоков.
- Создайте Verilog-файл для BDF верхнего уровня под именем lab4_2.v, как вы это делали в Lab3.
- **Compile -> Compile All**, чтобы скомпилировать все файлы и исправить все ошибки. Используйте ModelSim, чтобы убедиться, что ваша схема работает правильно.

Входные сигналы сформируйте следующим образом:

- Сигнал **M**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 400 ns. Данная последовательность должна периодически повторяться с периодом 800 ns до конца времени моделирования.
- Сигнал **P**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 200 ns. Данная последовательность должна периодически повторяться с периодом 400 ns до конца времени моделирования.
- Сигнал **T**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 100 ns. Данная последовательность должна

периодически повторяться с периодом 200 ns до конца времени моделирования.

- Сигнал **h**: необходимо этому сигналу присвоить значение логического нуля на нулевой наносекунде и логической единицы на 50 ns. Данная последовательность должна периодически повторяться с периодом 100 ns до конца времени моделирования.

5. Задание

5.1. Реализуйте схематическое проектирование для функции f своего варианта задания.
5.2. Выполните компилирование и симулирование вашего проекта
5.3. Реализуйте решение своего варианта задания (реализация функции f) с помощью таблицы истинности с использованием Verilog.
5.4. Выполните смешанное проектирование своего варианта задания.

Вариант I. $f = x_1x_2 + \bar{x}_2x_3$ (для пункта 5.4: $x_1 = z_1z_2 + z_3z_4$ $x_3 = z_1z_3 + z_2z_4$)

Вариант II. $f = x_1x_2 + x_2\bar{x}_3$ (для пункта 5.4: $x_1 = z_1z_3 + z_2z_4$ $x_3 = z_1z_2 + z_3z_4$)

Вариант III. $f = \bar{x}_1x_2 + x_2x_3$ (для пункта 5.4: $x_1 = z_2z_3 + z_1z_4$ $x_3 = z_2z_3 + z_3z_4$)

6. Содержание отчета

6.1. Цель работы.
6.2. Схема и таблица истинности для своего варианта задания
6.3. Код на Verilog для структурного и поведенческого стиля программирования.
6.4. Выводы по работе

7. Контрольные вопросы

7.1. Изучите код на Verilog, заданный в разделе 3. Объясните работу оператора always @
7.2. Прочитайте раздел 4 и заполните таблицу истинности для ***lab4_2***

Inputs				Outputs		
M	**T**	**H**	**P**	**E**	**F**	**AC**
0	0	0	0			
0	0	0	1			
0	0	1	0			
0	0	1	1			
0	1	0	0			
0	1	0	1			
0	1	1	0			
0	1	1	1			
1	0	0	0			
1	0	0	1			
1	0	1	0			

1	0	1	1			
1	1	0	0			
1	1	0	1			
1	1	1	0			
1	1	1	1			

7.3. Заполните таблицу истинности для шага ***lab4_1***

Farmer	Cabbage	Goat	Wolf	Alarm
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	